Assignment 3: Serverless Customer Review Processing Pipeline Using AWS Lambda

Data Intensive Computing

Sajjad Ahmad Faisal Zada Adil Shinwari Kashir Hasnain Joey Tran

29 June 2026

Contents

1 4.1 Test 1 – Preprocessing **2**

1.1 Objective . . . 2

1.2 Output . . . 3

1.3 Results . . . 3

2 4.2 Test 2 – Profanity Check **4**

2.1 Objective . . . 4

2.2 Implementation . . . 4

2.3 Code . . . 4

2.4 Output . . . 4

2.5 Results . . . 4

2.5.1 Test 2A – Profane Review . . . 4

2.5.2 Test 2B – Clean Review . . . 4

2.6 Conclusion . . . 5

3 4.3 Test 3 – Sentiment Analysis **5**

3.1 Objective . . . 5

3.2 Implementation . . . 5

3.3 Code . . . 6

3.4 Test 3A – Positive Sentiment Classification . . . 6

3.4.1 Output . . . 6

3.4.2 Results . . . 6

3.5 Test 3B – Negative Sentiment Classification . . . 6

3.5.1 Output . . . 6

3.5.2 Results . . . 6

3.6 Test 3C – DynamoDB Required Fields Validation 7
3.6.1 Output 7
3.6.2 Results 7
3.7 Conclusion 7

4 4.4 Test 4 – Impolite Review Counter 8
4.1 Objective 8
4.2 Implementation 8
4.3 Code 8
4.4 Output 8
4.5 Results 9
4.6 Conclusion 9

5 4.5 Test 5 – Automatic Ban Logic 9
5.1 Objective 9
5.2 Implementation 9
5.3 Code 9
5.4 Output 9
5.5 Results 9
5.6 Conclusion 10

6 5. Results 10
6.1 Summary of Results 11

7 6. Conclusion 11

8 References 11

1 4.1 Test 1 – Preprocessing

1.1 Objective

The objective of this test is to verify that the Preprocessing Lambda correctly processes customer reviews by performing tokenization, stop-word removal, lowercase conversion, and lemmatization before storing the processed review in the preprocessing Amazon S3 bucket. ## Implementation The Preprocessing Lambda is automatically triggered when a new customer review is uploaded to the raw Amazon S3 bucket. The Lambda extracts the `summary`, `reviewText`, and `overall` fields from the JSON review object. It then tokenizes the text using NLTK's `word_tokenize`, removes English stop words, converts all tokens to lowercase, and applies lemmatization using the `WordNetLemmatizer`. Finally, the processed review is written to the preprocessing S3 bucket to trigger the next Lambda function. All bucket names are retrieved dynamically from AWS Systems Manager (SSM) Parameter Store. ## Code

```python
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
def preprocess_text(text):
    tokens = word_tokenize(text.lower())
    stop_words = set(stopwords.words("english"))
    lemmatizer = WordNetLemmatizer()
    tokens = [
        lemmatizer.lemmatize(token)
        for token in tokens
        if token.isalpha() and token not in stop_words
    ]
    return tokens
```

1.2 Output

Figure 1: Figure 1. Test 1 – Preprocessing Integration Test

1.3 Results

- The customer review was uploaded successfully to the raw Amazon S3 bucket.
- The Preprocessing Lambda was triggered automatically.
- The review text was tokenized successfully.
- English stop words were removed correctly.
- All tokens were converted to lowercase.
- Lemmatization was successfully applied.
- The processed review was stored in the preprocessing Amazon S3 bucket.
- The original overall rating was preserved.
- All preprocessing validation checks passed successfully. ## Conclusion The Preprocessing Lambda successfully completed all required Natural Language Processing (NLP) tasks before forwarding the processed review to the next stage of the serverless pipeline. The integration test confirms that the preprocessing component operates correctly and meets the assignment requirements.

2 4.2 Test 2 – Profanity Check

2.1 Objective

The objective of this test is to verify that the Profanity Check Lambda correctly identifies offensive language in customer reviews while allowing clean reviews to continue through the serverless processing pipeline.

2.2 Implementation

The Profanity Check Lambda is triggered automatically when a preprocessed review is uploaded to the preprocessing Amazon S3 bucket. The Lambda reads the processed review, analyses both the `summary` and `reviewText` fields using the **profanityfilter** library, and determines whether the review contains offensive language. If profanity is detected, the customer's impolite review count is incremented in DynamoDB. When the count exceeds three, the customer is marked as banned. The processed review is then written to the next Amazon S3 bucket to trigger the Sentiment Analysis Lambda. All bucket and table names are retrieved dynamically from AWS Systems Manager (SSM) Parameter Store.

2.3 Code

```
from profanityfilter import ProfanityFilter

pf = ProfanityFilter()

summary_profane = pf.is_profane(summary)
review_profane = pf.is_profane(review_text)

is_profane = summary_profane or review_profane

if is_profane:
    increment_impolite_count(reviewer_id)
```

2.4 Output

2.5 Results

2.5.1 Test 2A – Profane Review

- The review containing offensive language was uploaded successfully.
- The Lambda correctly detected the offensive words.
- The `is_profane` flag was set to **True**.
- The customer's impolite review count was updated in DynamoDB.
- The processed review was forwarded successfully to the next stage.
- The integration test passed successfully.

2.5.2 Test 2B – Clean Review

- A clean review without offensive language was uploaded successfully.
- The Lambda correctly classified the review as non-profane.
- The `is_profane` flag remained **False**.

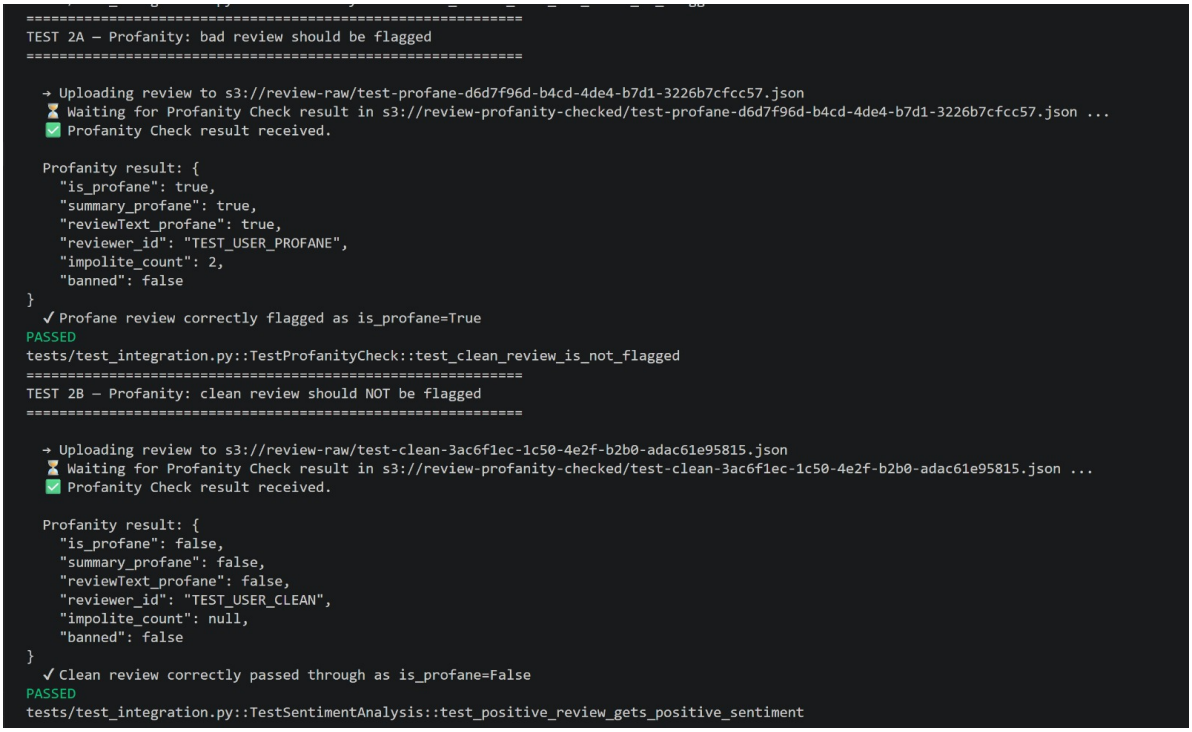

Figure 2: Figure 2. Test 2 – Profanity Check Integration Test

- The review continued to the Sentiment Analysis Lambda.
- The integration test passed successfully.

2.6 Conclusion

The Profanity Check Lambda successfully distinguished between offensive and non-offensive customer reviews. Reviews containing inappropriate language were correctly flagged and customer records were updated accordingly, while clean reviews continued through the serverless processing pipeline without interruption.

3 4.3 Test 3 – Sentiment Analysis

3.1 Objective

The objective of this test is to verify that the Sentiment Analysis Lambda correctly classifies customer reviews as **positive**, **negative**, or **neutral**, and stores the processed review together with all required attributes in the Amazon DynamoDB `review-results` table.

3.2 Implementation

The Sentiment Analysis Lambda is triggered automatically after the Profanity Check Lambda completes successfully. The Lambda reads the review from the S3 bucket and analyses the review text using the NLTK **SentimentIntensityAnalyzer (VADER)**. Based on the compound sentiment score, the review is classified as **positive**, **negative**, or **neutral**. Finally, the processed review and its metadata are stored in the DynamoDB `review-results` table for further analysis.

3.3 Code

```
from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

score = sia.polarity_scores(text)["compound"]

if score >= 0.05:
    sentiment = "positive"
elif score <= -0.05:
    sentiment = "negative"
else:
    sentiment = "neutral"
```

3.4 Test 3A – Positive Sentiment Classification

3.4.1 Output

Figure 3: Figure 3. Test 3A – Positive Sentiment Classification

3.4.2 Results

- A positive customer review was uploaded successfully.
- The Sentiment Analysis Lambda classified the review as **positive**.
- The sentiment label was stored successfully in the DynamoDB `review-results` table.
- The integration test passed successfully.

3.5 Test 3B – Negative Sentiment Classification

3.5.1 Output

3.5.2 Results

- A negative customer review was uploaded successfully.

Figure 4: Figure 4. Test 3B – Negative Sentiment Classification

- The Sentiment Analysis Lambda classified the review as **negative**.
- The sentiment label was stored successfully in the DynamoDB `review-results` table.
- The integration test passed successfully.

3.6 Test 3C – DynamoDB Required Fields Validation

3.6.1 Output

Figure 5: Figure 5. Test 3C – DynamoDB Required Fields Validation

3.6.2 Results

- The processed review was successfully stored in the DynamoDB `review-results` table.
- All required attributes were present in the stored record.
- The record contained the review ID, reviewer ID, ASIN, overall rating, sentiment label, and profanity flag.
- The integration test passed successfully.

3.7 Conclusion

The Sentiment Analysis Lambda successfully classified both positive and negative customer reviews and stored the processed results in DynamoDB. The validation test confirmed that all required attributes were saved correctly, demonstrating the successful integration of the sentiment analysis stage within the serverless review processing pipeline.

4 4.4 Test 4 – Impolite Review Counter

4.1 Objective

The objective of this test is to verify that the application correctly tracks the number of offensive reviews submitted by a customer and updates the `impolite_count` value in the Amazon DynamoDB table.

4.2 Implementation

The Impolite Review Counter is implemented within the Profanity Check Lambda. Whenever a review is classified as profane, the Lambda increments the customer's `impolite_count` in the DynamoDB table. This test submits three offensive reviews from the same reviewer and verifies that the counter increases correctly after each submission.

4.3 Code

```
table.update_item(
    Key={"reviewerID": reviewer_id},
    UpdateExpression="ADD impolite_count :inc",
    ExpressionAttributeValues={
        ":inc": 1
    }
)
```

4.4 Output

Figure 6: Figure 6. Test 4 – Impolite Review Counter

4.5 Results

- Three offensive reviews were submitted successfully by the same reviewer.
- Each review was correctly identified by the Profanity Check Lambda.
- The customer's `impolite_count` increased after every profane review.
- The final `impolite_count` stored in DynamoDB was **3**.
- The integration test completed successfully.

4.6 Conclusion

The Impolite Review Counter successfully tracked repeated offensive reviews and accurately updated the customer's record in DynamoDB. The successful execution of this test confirms that the application correctly monitors repeated policy violations before applying the automatic banning policy.

5 4.5 Test 5 – Automatic Ban Logic

5.1 Objective

The objective of this test is to verify that the application automatically bans a customer after submitting more than three offensive reviews by updating the `banned` status in the Amazon DynamoDB table.

5.2 Implementation

The Automatic Ban Logic is implemented within the Profanity Check Lambda. After each profane review, the customer's `impolite_count` is retrieved from DynamoDB. If the count exceeds three, the Lambda updates the customer's record by setting the `banned` attribute to **True**. This test submits a fourth offensive review from the same customer to verify that the banning policy is enforced correctly.

5.3 Code

```
if impolite_count > 3:
    table.update_item(
        Key={"reviewerID": reviewer_id},
        UpdateExpression="SET banned = :status",
        ExpressionAttributeValues={
            ":status": True
        }
    )
```

5.4 Output

5.5 Results

- A fourth offensive review was submitted successfully by the same reviewer.
- The Profanity Check Lambda detected the offensive review.
- The customer's `impolite_count` exceeded the allowed threshold.
- The customer's `banned` status was updated to **True** in DynamoDB.
- The processed review continued through the remaining stages of the pipeline.
- The integration test completed successfully.

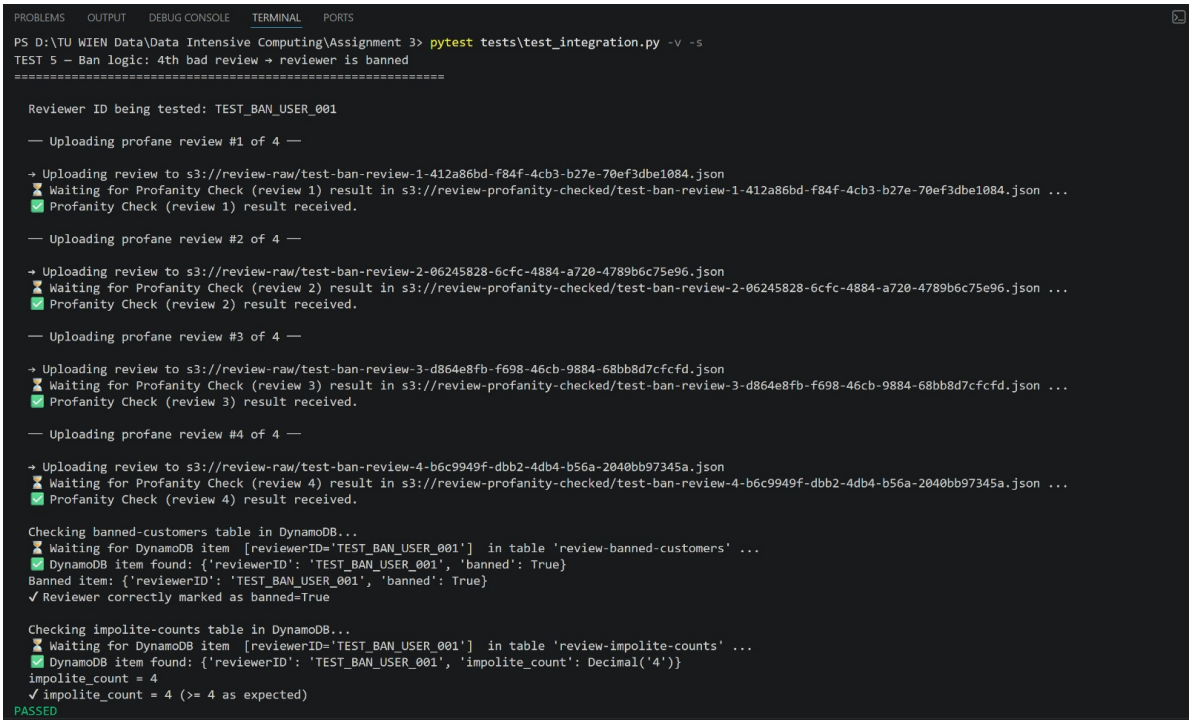

Figure 7: Figure 7. Test 5 – Automatic Ban Logic

5.6 Conclusion

The Automatic Ban Logic correctly enforced the application's moderation policy by banning customers who repeatedly submitted offensive reviews. The successful execution of this test confirms that the banning mechanism is fully integrated with the profanity detection workflow and DynamoDB.

6 5. Results

The complete integration test suite was executed using the MiniStack environment to verify the functionality of the serverless customer review processing pipeline. All AWS Lambda functions communicated successfully through Amazon S3 event triggers, and the processed data was stored correctly in Amazon DynamoDB. The test results confirmed that each stage of the application performed as expected.

Test	Description	Status
Test 1	Preprocessing	Passed
Test 2A	Profane Review Detection	Passed
Test 2B	Clean Review Validation	Passed
Test 3A	Positive Sentiment Classification	Passed
Test 3B	Negative Sentiment Classification	Passed
Test 3C	DynamoDB Required Fields Validation	Passed
Test 4	Impolite Review Counter	Passed
Test 5	Automatic Ban Logic	Passed

6.1 Summary of Results

The integration testing demonstrated that the serverless application successfully processed customer reviews through every stage of the workflow. The preprocessing Lambda correctly cleaned and normalized the review text, the profanity check Lambda accurately detected offensive language and updated customer records, and the sentiment analysis Lambda classified customer reviews and stored the processed information in DynamoDB. The impolite review counter and automatic ban mechanism also operated correctly, ensuring that repeated violations were handled according to the system requirements. Overall, all integration tests passed successfully, confirming that the application satisfies the functional requirements of the assignment.

7 6. Conclusion

This assignment successfully implemented a serverless customer review processing pipeline using AWS Lambda, Amazon S3, Amazon DynamoDB, and AWS Systems Manager (SSM) Parameter Store. The application automatically preprocesses customer reviews, detects offensive language, performs sentiment analysis, tracks repeated policy violations, and stores the processed results in DynamoDB.

The successful completion of all integration tests demonstrates that the application is reliable, scalable, and capable of processing customer reviews automatically through an event-driven architecture. The modular design of the Lambda functions makes the system easy to maintain and extend with additional Natural Language Processing (NLP) features in the future.

8 References

1. Amazon Web Services. *AWS Lambda Developer Guide*. https://docs.aws.amazon.com/lambda/
2. Amazon Web Services. *Amazon S3 User Guide*. https://docs.aws.amazon.com/AmazonS3/
3. Amazon Web Services. *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/amazondynamodb/
4. Amazon Web Services. *AWS Systems Manager Parameter Store*. https://docs.aws.amazon.com/systems-manager/
5. Bird, S., Klein, E., & Loper, E. *Natural Language Toolkit (NLTK) Documentation*. https://www.nltk.org/